MATHEMATICAL PROOF-LANGUAGE DESIGN

Sorrel

Contract-Typed Proof Plans for Human-Scale Mathematics

Properties, residual obligations, and explicit evidence interfaces over Lean

---

The proposal. Let authors write the mathematical route. Elaborate it into a reusable proof plan whose type states exactly what remains to be established. Keep alternative sufficient assumptions, not just one successful proof. Discharge routine work with Lean, Leant, and checked computation without changing the authored statement. **The increment over round three.** A precise finite

support calculus, residual-interface extraction, a scope-safe integration design, and an executed miniature frontend. A property list is useful; a typed explanation of how it licenses the next mathematical step is more useful.

Prepared for Vladimir Reshetnikov

Fourth iteration of the Brainstorm campaign

6 September 2026

Evidence status: source-based design and written proofs; 31 executed finite-model tests; 7,680 exhaustive seed-subset/atom comparisons. No complete Sorrel-to-Lean frontend, Lean compilation of the supplied specimen, or productivity experiment is claimed.

Abstract

Sorrel is a proposed mathematical authoring language implemented first as an extension of Lean's elaboration discipline, not its trusted conversion rules. Its central object is a *contract-typed proof plan*: a mathematical argument with a fixed target, explicit construction choices, and an ordered interface of residual proof obligations. Completed plans produce ordinary Lean terms. Incomplete plans are conditional constructions, never silently published as proofs of their conclusions.

The two round-three syntheses strongly support same-object property views, behavioral contracts, bounded inference, and certificate-producing services, but also warn that another surface syntax without an authoring experiment would add little. Sorrel therefore concentrates on the missing operational connection between these features. A finite, grounded implication engine computes antichains of sufficient assumption sets and retains checked conditional derivations. This supports alternative proof routes, precise repair frontiers, and support-restricted methods. The paper proves soundness, termination, relative completeness, and a residual-frontier theorem for that finite calculus, while separating it from the harder problem of dependent Lean elaboration.

Worked reconstructions study a compact-support Cauchy-transform formula in ProveIt, exact integral Frey coefficients, sharp Fabius regularity, and a finite tensor-family construction in the Fermat repository. They expose locality, almost-everywhere reasoning, exact division, and dependent structure choices that a useful type discipline must preserve. The proposal also specifies behavior-aware Leant integration, a shared reification boundary for certified computation, and a measured route from library interfaces to possible native refinement types. An executable Python model and a generated, explicitly uncompiled Lean implication specimen accompany the article. Their scope is narrow and reproducible: they test the support calculus, not a finished proof assistant.

Contents

1 A fourth-round decision, not a tenth spelling

1.1 The central idea

A mathematician's instruction "apply the fundamental theorem of calculus" usually identifies an argument more clearly than a sequence of coercion, continuity, and interval-restriction steps. It does not eliminate those steps. The interesting language problem is to make them *generated consequences of a stated mathematical route*, with a useful explanation when they fail.

Sorrel makes that route a typed object. A plan consists of a fixed mathematical request, a sequence of author-selected steps, and the premises needed to complete those steps. A plan may be reused under another context by supplying its premises. Its intermediate objects remain available to later obligations. Its completed evidence is an ordinary Lean term, together with a separately checkable record of any required method policy.

For example, a quotient-transport plan should not merely report that it cannot find a proof. It should report:

> To identify this integer quotient with this rational quotient, establish exact divisibility. The selected parity route needs oddness of the exponent. An alternative registered route accepts a direct divisibility proof. The target coefficient and its two coefficient domains have not changed.

This is neither automatic hypothesis strengthening nor a claim that oddness is mathematically necessary. It is an interface describing sufficient ways to complete a particular argument.

1.2 What is inherited and what is proposed here

The round-three reports *From Properties to Proof Obligations* and *Nine Refinements* already recommend fixed-object refinements, explicit structures, dependent behavior specifications, bounded proof-producing inference, exact-target replay, and an obligation frontier [1, 2]. Sorrel adopts these decisions. Renaming a property row, repeating the type of a hypothetical checker, or replacing `have` by a new keyword would not be a substantive fourth iteration.

Sorrel's proposed increment is the following combination:

1. A proof plan exports its *residual telescope*, not just a failure message. The same object can drive author assistance, Leant completion, and construction of a reusable helper theorem.
2. A precisely delimited inference layer retains *alternative minimal supports* within its registered rule fragment. This makes an explanation stable under some edits and reveals alternative repairs without asserting unrestricted weakest preconditions.
3. Method restrictions are checked over typed plan structure, independently of proof irrelevance and kernel truth. Support policies are applied before support minimization, so an unacceptable shortcut cannot suppress a permitted argument.
4. A small executable model exercises the inference and residual-interface mechanisms now. The full frontend and its evaluation remain explicit next artifacts, not achievements inferred from the model.

These are integration proposals relative to this campaign, not claims of historical invention. The support calculus is closely related to assumption-based truth maintenance and provenance annotations [13, 14]. The intended contribution is fitting such reasoning into a dependent, proof-producing mathematical authoring interface without confusing conditional support with established truth.

1.3 A realistic definition of concision

There are at least four different quantities: authored mathematical decisions, authored formal bookkeeping, generated proof size, and reader effort. A system can reduce one while increasing the others. A three-line call to a hundred-line new helper theorem may be excellent library engineering; it is not by itself evidence for a new language.

Sorrel therefore seeks to reduce repeated *author interventions* while preserving recoverable mathematical meaning. The selected domain, measure, branch, witness, quantifier order, and strength of conclusion should remain visible. Proof arguments for consequences already established by the context may be folded. The expanded view must explain them using the actual accepted plan, not a plausible narrative generated after the fact.

2 Reading the corpus critically

2.1 Sources and the limits of this review

Both main round-three TeX reports were examined, including their reconciliation, worked examples, evidence qualifications, and fourth-iteration questions. The design also uses direct readings of four mathematical source files and Leant's verification interface. Table 1 fixes the source identities. This is not an audit or rebuild of either mathematical repository, nor a claim to have independently reread all nine original proposals. Attribution to an individual round-three idea below is mediated by the two syntheses unless otherwise indicated.

Table 1: Source snapshots used in this article. Complete commit and blob identities are recorded in `sources.json`.

Repository	Commit prefix	Directly inspected material
Brainstorm	`bf333390`	Both round-three main TeX syntheses.
ProveIt	`24ce8bd7`	`CauchyCDF.lean`; `Regularity.lean`.
Fermat repository	`aa2d8b34`	`Def_FLTPrelim_FreyPackage.lean`; finite tensor-family solution.
Leant	`3a40904b`	`src/Leant/Synth/Verification.hs`; public synthesis and suggestion descriptions.

The revised syntheses additionally discuss Leant at commit `823259f7`. That account is useful, but it is not silently substituted for the `3a40904b` reference resolved by the connector in this review. Claims about the later behavioral-decision pipeline are attributed to the reports; the callback-receipt distinction is independently visible in the inspected Haskell source [7, 1, 2].

The reports describe successful Lean checks of seven earlier semantic cores, including 49 named theorem declarations; the second gives 48 axiom inventories. Those are their execution results, not fresh runs here. In particular, Moraine's checked length-interpreter theorem must not be inflated into a checked implementation of its coefficient normalizer. Schist's checked affine example is a useful narrow end-to-end proof chain, not a general CAS. The reports' deliberately failing `mvcgen` probe is not a clean whole-file compilation [1, 2].

2.2 The important corrections must survive round four

The synthesis process corrected several attractive but false simplifications. Positive certificate soundness does not require decision completeness. An abstract counterexample requires realization and the correct direction of specification transfer. Over `List Empty`, different affine length coefficients may describe the same concrete behavior. Seven similar Lean encodings are not

literally alpha-equivalent. And finiteness of an index set does not imply that its algebra factors are finite modules [1, 2].

These corrections influence Sorrel's interfaces. A service advertises separate positive, negative, and completeness contracts. A plan records its actual input domain. A dependent construction exports the structures on its chosen witness. An explanation says "this registered route requires" rather than "the theorem requires" unless necessity has separately been proved.

2.3 Where actual Lean verbosity occurs

The ProveIt Cauchy–CDF proof derives nonzero denominators from exclusion of a complexified interval, obtains continuity and integrability, proves a primitive identity on subintervals, transports it almost everywhere under a support hypothesis, and applies a layer-cake identity. The argument is mathematically coherent; much of its length connects these interfaces [3].

The Fabius regularity file similarly connects a differential equation, nonnegative range bounds, real norms, nonnegative-real constants, and a Lipschitz theorem. It separately proves optimality using a derivative value. Its short corollaries are controls that Sorrel should leave short [4].

The Frey file already uses a bundle containing mathematical properties. Lean is not missing the ability to represent such information. The problem is making operation-specific consequences available without repeatedly unpacking and adapting the bundle. Its two displayed curve definitions also show why identical notation across $\mathbb{Z}$ and $\mathbb{Q}$ can hide different division semantics [5].

Finally, the tensor-family proof constructs a type, ring and algebra structures, finite/free proofs, a family of equivalences, and a naturality law. These objects have dependent types: they cannot be represented faithfully as an unordered bag of adjectives [6].

2.4 The strongest existing baseline

Mathlib's `fun_prop` already decomposes function properties, uses local hypotheses, supports registered transition rules, and exposes side-condition dischargers. Its documentation explicitly warns about the search cost of transition rules [9]. Sorrel must use and compare against this capability rather than present continuity inference as a new feature.

Version discipline also matters. The corpus reports use Lean 4.32.0; the August 10, 2026 release notes for Lean 4.33.0 describe further automation and incremental elaboration changes, including the newer `vcgen` name for `mvcgen'`. This does not retroactively change the pinned source experiment. A modern comparison should specify its toolchain and include applicable verification-condition generation, theorem search, and arithmetic tactics [12].

3 The author-facing language

3.1 Write the argument, not an unbounded invitation to search

Sorrel's mathematical surface is deliberately small. It has fixed variables, assumptions, definitions, assertions, witness constructions, case splits, induction, chains of equalities or inequalities, and named proof methods. Ordinary Lean expressions and proofs remain escape hatches. Unrestricted natural-language interpretation is not the acceptance boundary.

The following is *proposed notation*, not syntax accepted by the supplied finite-model parser:

```
theorem reciprocal_integral (mu : FiniteMeasure Real)
    (a b : Real) (z : Complex)
    assuming a <= b, ae_support mu [a,b], z outside complexify [a,b]:
  integral x under mu of 1/(z-x)
    = mass(mu)/(z-b)
      - integral t from a to b of closed_cdf(mu,t)/(z-t)^2
proof
  let r(t) := 1/(z-t)
  let k(t) := r(t)^2
  have r'(t) = k(t) on [a,b] by differentiate
  have integral t from x to b of k(t) = r(b)-r(x)
    for x in [a,b] by fundamental_theorem
  use closed_cdf_layer_cake with mu, k
  replace the inner integral almost everywhere using the primitive
  conclude by linearity and additive algebra
```

The surface retains the actual mathematical decisions: the primitive, layer-cake method, almost-everywhere replacement, and final rearrangement. It delegates denominator nonzeroness, subinterval restrictions, and standard continuity/integrability consequences. A proof of the whole theorem obtained by unrelated search would not count as following these required steps.

The notation `FiniteMeasure Real` is an author-facing package convention. At a Lean boundary it can elaborate to an ordinary measure and its `IsFiniteMeasure` instance. The measure is not replaced, and the proof of finiteness is not a second measure. Likewise, `closed_cdf` in this finite-measure theorem denotes unnormalized cumulative mass, not a probability CDF with an implicit normalization.

3.2 Four verbs with different logical effects

`assume P` extends a local proof context. It cannot be used at top level to discharge a pre-existing obligation without making the resulting theorem conditional. `have P by m` requests a proof of the fixed proposition P. `let x := e` fixes an object. `choose x satisfying S` delegates a construction under the already elaborated specification S.

This distinction prevents a common ambiguity: learning that an existing triple is primitive is not the same action as dividing the triple by its gcd. The latter constructs a different triple and owes a relation between the old and new witnesses. Sorrel gives those actions different source nodes.

3.3 Proof plans and holes

A plan can be named before all its conditions are filled:

```
plan cast_exact_quotient (n d q : Int)
  requires balance : d*q = n
           denominator : (d : Rat) != 0
  establishes (q : Rat) = (n : Rat)/(d : Rat)
  by map_balance_then_cancel
```

Here the input q is fixed. Another plan may construct it from $d \mid n$, with the zero-denominator policy explicit. The two plans are different APIs. At use sites, `requires` arguments are proof-implicit: the elaborator looks for evidence or leaves a typed goal. It does not ask a global instance search to interpret every proposition as a typeclass.

A user can inspect a plan's residual interface, fill one obligation explicitly, ask Leant to assemble a remaining adapter, or export the conditional plan as a helper theorem. Exporting the conditional

theorem does not establish an unconditional target. The source view must keep that distinction visible.

3.4 Hints, requirements, and publication

A name written `by? arithmetic` is a search hint. A name written `by arithmetic using only H` selects a required method and support policy. This is an illustrative distinction; a concrete parser may choose different punctuation, but the semantic difference is mandatory.

Every displayed assertion receives an evidence node. If the final theorem happens not to use an earlier `have`, the earlier assertion must still be checked before the document is marked complete. Unfinished assertions remain visibly unfinished. A prose comment may explain intuition, but it is not rendered as an established mathematical claim merely because it looks like one.

4 A property- and behavior-aware type discipline

4.1 Three contexts, one logical authority

For explanation, distinguish a data context Δ, a proof context Γ, and a service profile $\mathcal{R}$. The first fixes objects, universes, operations, measures, domains, and any authorized construction parameters. The second contains ordinary Lean hypotheses and accepted proofs. The third selects registered theorem schemas, automation budgets, and method policies. In an implementation, Δ and Γ are classifications of entries in Lean's dependent context, not competing logical environments. Their dependency order must be retained.

A property view is written

$$\Delta; \Gamma \vdash t : A \mid \{P_1(t), \ldots, P_k(t)\}.$$

Its meaning is simply that $t : A$ is fixed and the context contains evidence of each displayed proposition. The predicates may mention several objects; for example, $\mathsf{InversePair}(f, g)$, a naturality square, or $f =_{\mu\text{-a.e.}} g$. A row is a presentation of propositions, not a restriction to unary properties or a new form of mathematical truth.

There are four elementary operations. Introduction adds $p : P(t)$. Intersection combines evidence about the same t. Consequence applies an explicit implication $P(t) \to Q(t)$. Equality transport uses an actual proof $t = u$, or ordinary definitional equality, to obtain the corresponding property of u. None selects a different underlying multiplication, topology, measure, or branch of a function.

4.2 Property-implicit binders are a real source-level extension

Consider a method of source type

$$\begin{aligned}&\mathsf{quotientDerivative} : \Pi f, g, f', g', x,\\ &\quad \{h_f : \mathsf{HasDerivAt}(f, f', x)\} \to \{h_g : \mathsf{HasDerivAt}(g, g', x)\} \to\\ &\quad \{h_0 : g(x) \neq 0\} \to T(f, g, x).\end{aligned}$$

Braces here designate property-implicit parameters rather than ordinary implicit data inference. At application, Sorrel checks the data arguments, instantiates the exact required propositions, obtains evidence through the selected profile, and inserts the proof arguments. When a proof is unavailable, the application remains an open plan with that parameter in its interface.

This changes the author's typing discipline: uses of an operation are checked against mathematical properties and behavior specifications. Yet the result is an ordinary dependent application accepted by Lean. Calling it a type-system extension is appropriate at the source language level; calling it a new Lean kernel rule would be inaccurate.

4.3 Local views versus exported types

At an API boundary, a refined value can be represented by

$$\{x : A \mid P(x)\},$$

or a structure with a value and proof fields. Lean's existing `Subtype` already has this interpretation and is represented like the carrier in compiled code. Proof irrelevance also avoids distinguishing subtype values merely because their proof fields differ [10].

Locally, repeatedly repackaging t is unnecessary. A view attaches evidence to the already elaborated term. This retains convenient interaction with existing APIs and avoids inventing chains of wrappers just to add more known properties. The benefit to measure is fewer elaboration interventions, not an unsupported claim that Lean's proof fields incur runtime storage costs.

Property polymorphism can be expressed without a new kind of row variable. For instance, an interface parameter $\rho : A \to \mathsf{Prop}$ can represent an open family of caller-supplied properties. An operation that preserves that family must have a theorem stating so. Sorrel must not infer arbitrary preservation from a syntactically polymorphic type. In particular, a type parameter does not by itself supply a parametricity theorem in classical Lean; the round-three analyses correctly retain this warning [1, 2, 11].

4.4 Behavior is a relation involving the input and output

For a fixed total function $f : A \to B$, define

$$\mathsf{Beh}(f; P, Q) := \forall x : A,\ P(x) \to Q(x, f(x)), \qquad Q : A \to B \to \mathsf{Prop}.$$

This represents, for example, a sorting function whose output is both ordered and a permutation of *its actual input*. A postcondition only about output length is weaker. A constant list can satisfy length and orderedness while losing the input elements; a constant empty list can satisfy orderedness alone.

Consequence is proof-producing:

$$\frac{\mathsf{Beh}(f; P, Q) \quad P' \Rightarrow P \quad \forall x, y,\ P'(x) \to Q(x, y) \to Q'(x, y)}{\mathsf{Beh}(f; P', Q')}.$$

Thus a client supplying stronger premises can use an existing contract and accept a weaker conclusion. Conversely, an implementation substituted for a client's expected operation must demand no more and guarantee no less. These are the two views of the same implication directions.

Proposition 4.1 (Composition retains the intermediate)**.** *Let $f : A \to B$, $g : B \to C$, and suppose $\mathsf{Beh}(f; P, Q)$, $\mathsf{Beh}(g; U, V)$. If*

$$P(x) \to Q(x, y) \to U(y),$$
$$P(x) \to Q(x, y) \to V(y, z) \to W(x, z),$$

then $\mathsf{Beh}(g \circ f; P, W)$.

Proof. Fix x and $h : P(x)$. The first contract gives $Q(x, f(x))$. The first bridge gives $U(f(x))$; the second contract then gives $V(f(x), g(f(x)))$. Applying the second bridge proves the conclusion. □

A dependent operation may require the intermediate value, its index, or a proof obtained at that stage. Its interface must keep those binders in order. Flattening $U(f(x))$ into a string called "the next guard" loses the information needed for a well-typed application.

4.5 Whole-function and higher-order constraints

Many mathematical behaviors are not one-input/one-output postconditions. Continuity, monotonicity, linearity, naturality, inverse-pair laws, and prefix locality quantify over multiple evaluations or entire families. Sorrel accepts arbitrary Lean propositions about the operation:

$$\mathsf{RelSpec}(f), \qquad \forall x, y,\ R(x,y) \to S(f(x), f(y)), \qquad \forall D, E, u,\ \mathsf{Commutes}(f_D, f_E, u).$$

The inference profile supports only specified theorem shapes, but the type language does not reduce these properties to output tags.

Higher-order composition follows the same rule. To synthesize an adapter on invariant subtypes, the provider must prove that the original map preserves the predicate. To lift a family to a natural transformation, the provider must prove the commuting square. A matching arrow type provides neither theorem.

4.6 Partial domains, totalized operations, and execution

There is a semantic difference between

$$f : A \to B \quad \text{with} \quad \mathsf{Beh}(f; P, Q) \quad \text{and} \quad g : \{x : A \mid P(x)\} \to B.$$

The first is total and has a conditional theorem; the second requires a refined input. Neither licenses an arbitrary extension of g to all of A. Sorrel preserves imported Lean operations, including totalized division and derivatives. A domain-sensitive mathematical API is an explicit new interface, not a hidden reinterpretation of those operations.

Effectful programs need an operational semantics or an existing Hoare-style specification framework. A stateful routine's contract may quantify over initial and final states, exceptions, and termination. The surface can expose these conditions, but the pure Beh connective is not a substitute for that semantics. Similarly, a stated cost bound must refer to a defined cost model and be proved; a search timeout is an operational outcome, not a mathematical precondition or complexity theorem.

4.7 A native refinement former: worth considering, not assuming

A later kernel experiment could introduce a primitive refinement former $\mathsf{Ref}(A, P)$ with explicit introduction (a, p), erasing projection val, a property projection, and proof-directed subsumption:

$$\frac{a : A \quad p : P(a)}{(a,p) : \mathsf{Ref}(A,P)}, \qquad \frac{u : \mathsf{Ref}(A,P) \quad h : \forall x, P(x) \to Q(x)}{\mathsf{weaken}(h,u) : \mathsf{Ref}(A,Q)}.$$

These rules have a straightforward candidate translation into existing Lean subtypes. They might improve elaboration or transport handling in some workloads, but they do not add the ability to express the properties discussed here. Nor does the translation alone prove the implementation of a new kernel sound.

A serious native design must specify universes, dependent substitution, conversion, proof irrelevance, recursors, and the interaction with elimination from Prop. It must also preserve the distinction between existence in Prop and executable data. Obtaining a witness through classical choice is a different guarantee from constructing an executable algorithm.

The proposed decision gate is concrete: profile conservative Sorrel, isolate a transport or checking bottleneck, implement the smallest native alternative, and compare proof-term growth and checking cost. Independent kernel-checker work such as Lean4Lean is relevant background for such an experiment [16]. Refinement calculi with carefully restricted checking rules also show that the design space is not exhausted by unrestricted semantic subtyping [15].

Sorrel rejects *open-ended semantic entailment in conversion* as its default. Inserting a checked implication application is predictable at the kernel boundary; asking conversion to discover arbitrary implications is a much stronger design commitment. CAS normalization likewise belongs in a proof-producing step unless a separate, precisely specified extension has its own metatheory and measurements.

5 Residual interfaces: the type of an unfinished argument

5.1 A residual telescope is not a set of suggestions

An ordered telescope has the form

$$\Theta = (h_1 : G_1,\ h_2 : G_2(h_1), \ldots, h_k : G_k(h_1, \ldots, h_{k-1})).$$

A selected plan elaborates to

$$\Delta; \Gamma \vdash \pi \rightsquigarrow [\Theta\ ;\ e : T], \qquad \Delta, \Gamma, \Theta \vdash_{\text{Lean}} e : T.$$

For a theorem request, T is the translation of the frozen authored proposition. The residual telescope normally contains proof parameters. Authorized data construction is represented explicitly by a construction node or a dependent result type; it is not disguised as a routine proof hole.

Abstraction yields a conditional term

$$\lambda h_1 \ldots h_k . e : \Pi\Theta,\ T.$$

Supplying a well-typed substitution for Θ yields a proof of T. Without that substitution, the plan proves only the conditional statement. This is the exact boundary behind the user-facing word "open."

5.2 Composition is typed substitution

Suppose the next plan requires $u : U$, and a previous plan constructs $e : U$ under residual telescope Θ. Replace u by e in the next plan and its later obligations, append only dependencies that are well-formed after that substitution, and recheck the resulting telescope. Duplicate proposition obligations can be shared when their elaborated types match; merely identical text is insufficient.

Consider an exact quotient. A construction may introduce $q : \mathbb{Z}$ and a proof $dq = n$. The later cast step depends on *that* q. An obligation about an independently synthesized q' cannot discharge it just because both objects are described as "the quotient." Either prove the appropriate equality or retain the correct witness from the beginning.

Proposition 5.1 (Conditional elaboration invariant)**.** *For the restricted plan rules of property introduction, theorem application, explicit transport, dependent substitution, and case/induction application, assume that their Lean translations and data elaborations are well-typed. If a plan has residual interface* $[\Theta; e : T]$*, then its conditional translation has type* $\Pi\Theta, T$*. Any well-typed filling of* Θ *produces a term of the same translated target* T*.*

Proof. Property introduction inserts its supplied proof. A method application uses its registered theorem with the elaborated data and proof arguments. Transport applies the supplied equality eliminator or relation-specific consumer. Composition uses substitution into a well-formed dependent context. Cases and induction apply their corresponding eliminators, with all branch obligations abstracted in the scopes where they are valid. Induction over the plan derivation gives a well-typed e under Θ; repeated lambda introduction and substitution give the two conclusions. □

This is a paper invariant for the stated rules, not verification of a parser, unifier, incremental elaborator, or arbitrary tactic. Its value is identifying what such components must preserve, rather than claiming that kernel-checking whatever they emit proves source fidelity.

5.3 The semantic header is fixed before proof search

The header contains the requested proposition, its free data parameters, selected structures, quantifier order, and result relation. For example, "equal almost everywhere under μ" and "equal at x" have different headers. So do a bound and a least bound, or an arbitrary root and the root in a specified interval.

Sorrel classifies unresolved metavariables before launching a provider. Meaning-bearing data must be resolved from the source or from an explicitly authorized construction specification. Proof metavariables remain obligations. A provider is not allowed to instantiate an unknown measure, domain, or coefficient structure merely because doing so makes the proof easy.

In a first Lean implementation, the header should be stored as the elaborated expression with its context and universe instantiations. After provider execution, the implementation checks that no protected assignment has changed. A pretty-printed digest is useful for display and lookup but does not replace the expression-level comparison. Source meaning is not preserved merely by hashing the wrong expression consistently.

5.4 Residual interfaces as reusable mathematical lemmas

When an author repeatedly uses a plan, Sorrel can propose extracting a helper whose parameters are the retained data and residual premises. This is a mechanical interface extraction from an accepted conditional argument, not a claim to have discovered the weakest possible theorem.

The Frey example motivates this sharply. A helper about a_4 should not inherit oddness and the residue of a when its chosen proof only uses $2 \mid b$ and $p \geq 4$. Conversely, a theorem-level extraction must also account for the dependencies of any called helper. The system cannot decide support by searching for local hypothesis names in a pretty-printed proof.

A conservative implementation extracts the helper in a deliberately restricted context and asks Lean to check it there. This establishes a reusable interface with the permitted assumptions. It does not establish mathematical indispensability of those assumptions, nor semantic consistency of the context. A benchmark that is meant to exercise arithmetic should separately supply an inhabitant of its premises.

5.5 Why a method record is not just a proof term

Lean's proof irrelevance makes proofs of one proposition logically interchangeable; it is not a record of the author's intended argument [11]. A proof term can be inspected operationally, but occurrence of a theorem constant is not enough to show that a prescribed method actually justified a step. A provider could insert a decorative use and then conclude by an unrelated contradiction.

Sorrel's strict method mode therefore retains an *intensional plan certificate*. Its nodes apply approved theorem schemas to specified data and premise nodes. The plan checker verifies the requested local conclusions, allowed rules, and dependencies; Lean checks the resulting proof. A policy may authorize arbitrary Lean proofs in designated leaves, or may require those leaves to be closed in a restricted module. Those two policies are not equivalent and must not share a misleading badge.

This does not solve the philosophical problem of recognizing the same human argument in all equivalent proofs. It defines a narrow, inspectable guarantee: the submitted plan belongs to

this accepted grammar and supplies this fixed target using these allowed theorem interfaces. A kernel-correct proof that fails the method policy is reported as such, not called mathematically false.

6 Alternative supports with a precise finite calculus

6.1 The deliberately limited automatic fragment

Fix a well-formed context and a finite universe $\mathcal{Q}$ of ground propositions. "Ground" means that the objects, structures, and other data arguments have been fixed as elaborated expressions. Local variables such as an arbitrary real x are allowed; there is no enumeration of real numbers.

Let $H = \{H_1, \dots, H_m\} \subseteq \mathcal{Q}$ be designated assumption slots. They are possible *inputs to a conditional plan*, not new axioms accepted as true. A subset $K \subseteq H$ records slots currently supplied with actual evidence. Fix a finite rule table $\mathcal{R}$, whose rules are

$$r: \ P_1 \wedge \cdots \wedge P_k \Longrightarrow Q, \qquad P_i, Q \in \mathcal{Q}.$$

In the intended Lean integration, every rule instance has a checked theorem term. In the accompanying Python model, rules are stipulated implications, so its certificates are conditional on that rule table.

Definition 6.1 (Supports and labels)**.** A support of Q is a set $S \subseteq H$ for which a finite rule derivation of Q exists with assumption leaves drawn from S. The label $\mathcal{L}(Q)$ is the collection of inclusion-minimal supports. A retained witness records a finite derivation using exactly its leaf set.

Minimality is inclusion-minimality inside this fixed vocabulary and rule profile. It is not minimum cardinality, minimum effort, weakest semantic precondition, or a claim that another library theorem cannot prove more.

6.2 The algorithm

Initialize $\mathcal{L}(H_i)$ with $\{H_i\}$, and every other label with the empty family. A zero-premise rule contributes the empty support. For a rule $P_1, \dots, P_k \Rightarrow Q$, combine retained supports of its premises:

$$\mathcal{L}(Q) \leftarrow \operatorname{Min}\left(\mathcal{L}(Q) \cup \{S_1 \cup \cdots \cup S_k : S_i \in \mathcal{L}(P_i)\}\right), \tag{6.1}$$

where Min discards strict supersets. A newly inserted support receives the proof tree formed from its premise witnesses and the actual rule. Repeat until no label changes.

For instance, suppose the registered routes include

$$E \wedge L \Rightarrow D_{16}, \qquad E \wedge L \wedge O \wedge A \Rightarrow D_4, \qquad D_4^{\text{direct}} \Rightarrow D_4.$$

For the joint goal $D_4 \wedge D_{16}$, the minimal supports are

$$\{E, L, O, A\}, \qquad \{E, L, D_4^{\text{direct}}\}.$$

If E, L, A already have evidence, the open frontier is either O or a direct proof of D_4. Neither is installed as an assumption of the final theorem without the author's explicit action.

6.3 Soundness and termination

Theorem 6.2 (Support soundness)**.** *Every support retained by* (6.1) *has a finite derivation from exactly its recorded assumption leaves. If the rules are interpreted by Lean proofs, that derivation translates to a conditional Lean proof of its conclusion.*

Proof. An initial support is witnessed by its own assumption. The empty support of a zero-premise rule is witnessed by that rule. Every subsequent witness applies a rule to already existing finite witnesses. Its leaf set is the union of theirs, which is precisely the inserted support. Removing dominated supports does not alter any retained witness. The Lean interpretation follows by theorem application and abstraction over the leaves. □

Theorem 6.3 (Finite termination)**.** *For fixed finite $\mathcal{Q}, H, \mathcal{R}$, saturation terminates. At most $|\mathcal{Q}|2^m$ distinct support insertions can occur.*

Proof. For each proposition, regard its label as representing the upward-closed family of all supersets of its elements. An insertion strictly enlarges that family; removing strict supersets during minimization changes no represented support. A removed support can never be reinserted, since some retained subset continues to dominate it, possibly through an even smaller replacement. There are only 2^m candidate subsets per proposition. Each complete pass that does not terminate makes at least one such insertion. □

The bound counts successful label changes, not rule attempts or Lean cost. Repeated Cartesian products, theorem instantiation, normalization, checking, and witness construction are additional work. Even the number of incomparable supports can be exponential: a family of all $\lfloor m/2 \rfloor$-element subsets has $\binom{m}{\lfloor m/2 \rfloor}$ members. This is a reason for keeping the exact mode small, not for concealing the worst case.

6.4 Relative completeness and schedule independence

Theorem 6.4 (Exactness of the final labels)**.** *After saturation, a set $S \subseteq H$ derives Q by the fixed rule table if and only if it contains some $U \in \mathcal{L}(Q)$. Consequently the labels are exactly the minimal supports and are independent of fair scheduling.*

Proof. The reverse implication follows from Theorem 6.2 and weakening. For the forward implication, use induction on the height of a finite S-supported derivation. An assumption leaf has a retained support contained in its singleton; an unconditional rule has empty support. For a rule node, each premise derivation has a retained support contained in S by induction. At saturation, their union either belongs to the conclusion's label or is dominated by a retained smaller support. Thus the conclusion has a support contained in S. The collection of inclusion-minimal deriving sets is uniquely determined by the rule table, so any saturated fair schedule yields that same collection, although witness terms may differ. □

Unseeded cycles such as $P \Rightarrow Q \Rightarrow P$ create no proof. Once a seed exists, any retained proof still has a finite tree. This is not coinductive reasoning. Deleting a seed and rerunning the algorithm cannot justify the former result by its own cyclic dependency.

6.5 The residual-frontier theorem

Define

$$\mathcal{F}_K(Q) = \mathrm{Min}\{S \setminus K : S \in \mathcal{L}(Q)\}.$$

An empty element means a registered route is already supported. An empty *family* means no route has been derived in the fixed fragment. These must have different displays.

Theorem 6.5 (Sufficient repairs, exactly within the fragment)**.** *For any set $U \subseteq H \setminus K$ of additional supplied assumptions, Q is derivable from $K \cup U$ exactly when some $F \in \mathcal{F}_K(Q)$ satisfies $F \subseteq U$.*

Proof. By Theorem 6.4, derivability is equivalent to the existence of $S \in \mathcal{L}(Q)$ with $S \subseteq K \cup U$. This is equivalent to $S \setminus K \subseteq U$. Removing dominated residual sets preserves that existential condition. □

The theorem formalizes a useful editor operation: after removing a supplied hypothesis, show the remaining ways to complete the same selected request. A frontier element is a sufficient set of proof obligations, not permission to assert them. If their proofs are hard or false in the current mathematical context, the theorem remains open. The engine has not decided otherwise.

6.6 Budgets, policies, and dependent boundaries

A bounded run may retain only a sound subset of the final label families. Its successful conditional proofs remain valid, but its frontier is incomplete. An empty partial frontier proves neither impossibility nor absence of another registered route. Sorrel reports the support inventory's completeness separately from the existence of a checked proof.

Rules must be filtered for a required-method policy *before* computing labels, or labels must be indexed by a sufficiently precise policy state. Suppose an unrestricted contradiction shortcut supplies a smaller support than an allowed arithmetic argument. Discarding the arithmetic support first and filtering afterward can erase the very route the user requested. The supplied model reruns saturation with the allowed rule table; it does not attempt a global policy automaton.

Finally, this calculus is not a solver for arbitrary dependent telescopes. Its atoms live in one fixed, well-formed context. A premise involving a new existential witness, a branch-local variable, or an unresolved structure cannot be moved into the finite layer by giving it a string name. It first needs a typed plan node that introduces the witness or scope. Support labels are local to that node. This boundary is essential to the relevance of the finite correctness results.

7 Grounding, rewriting, and incremental scope

7.1 A finite rule set must actually be generated finitely

Theorem 6.3 assumes a finite ground rule table. It does not justify a frontend that first generates infinitely many theorem instances. Sorrel's first implementation therefore uses an explicit *grounding profile*. Its inputs are the selected operation's signature, the local context, a finite collection of eligible data terms, and a finite registry.

The initial term bank contains the request's subterms, named local objects, and explicitly supplied construction results. A profile may add terms through a whitelist of constructors to a fixed depth d. If there are initially M terms, at most C constructors of arity at most r, and no fresh unbounded literal generation, a crude finite bound is obtained from

$$T_0 = M, \qquad T_{j+1} = T_j + C \max(1, T_j)^r.$$

A schema with k eligible data parameters then has at most T_d^k candidate assignments before type filtering. Universe instantiations and structure arguments also come from explicitly finite choices.

Each assignment is checked against the theorem's actual dependent type. Metadata labels binder roles; it does not override them. Proof arguments become premise atoms. Schemas that require an unchosen data witness, higher-order synthesis outside the permitted fragment, or a new unresolved structure are not ground rules. They become explicit plan obligations or construction nodes.

This naive bound is intentionally pessimistic. A practical implementation uses backward reachability from the demanded conclusion, discrimination indices, and lazy generation. Its claim remains relative to the ground instances actually covered by the profile. "No supported instance generated" is a precise outcome; "no proof exists" is not.

7.2 Reuse property automation instead of copying it

The grounding layer should not replace `fun_prop`'s decomposition of function expressions. A registered provider can solve a property goal using `fun_prop` and return its proof as a leaf, while Sorrel tracks the operation-level dependencies that connect it to other kinds of obligations. A more explanatory provider may expose a validated decomposition into subgoals. Both modes are useful, but only the latter supports fine-grained frontier explanations through the provider's internal steps.

The same distinction applies to arithmetic. A successful `linarith` call can close a guard without teaching Sorrel every arithmetic inference. If the intended experiment is about explaining a removed inequality, the provider must expose enough certified dependency information or be rerun in restricted contexts. The extra explanatory work is a measurable cost.

7.3 Anchors are typed expressions, not identifiers

Suppose $p : P(t)$ is available and simplification changes a use site from t to u. There are three legitimate cases. Definitional equality permits ordinary reuse. A supplied equality $e : t = u$ permits transport. A weaker relation requires an appropriate consumer theorem. None is implemented correctly by matching normalized strings.

Sometimes only the proposition changes: a proof of $P(t) \leftrightarrow Q(u)$ permits a proof of $Q(u)$ without any claim that $t = u$. With dependent indices, both the term and the property family may need transport. Sorrel asks Lean to check the resulting proposition, retaining universes and chosen structure dictionaries. An unsupported transport shape remains an obligation.

The original fact need not disappear when a goal is rewritten. The index may keep it at its original anchor and insert a checked bridge at the consumer. This often avoids expensive global reindexing and preserves the explanation's connection to the author's original object.

7.4 Edits rebuild contexts; proofs do not migrate by hash

A source edit can replace local identifiers and change the context in which a term was elaborated. The initial Sorrel implementation gives the index the lifetime of a Lean elaboration snapshot. A rebuilt declaration reconstructs its index. Reuse across snapshots requires either re-elaboration from source or a checked mapping of local declarations and their dependent types.

A source hash or receipt locates an artifact. It is not a proof that two contexts are interchangeable. A renamed binder may be harmless; a different measure hidden behind the same printed symbol is not. The safe initial strategy is rebuilding, followed by selective cache improvements justified by measurements and exact rechecking.

The finite model implements a much smaller invariant: every atom includes a lexical context identifier, and a proof for one context is rejected in another. It does not implement Lean expression

transport, dependent index migration, or a real editor. That difference is tested and reported rather than hidden under the word "scope-safe."

8 Worked reconstruction: an atom-exact Cauchy formula

8.1 Statement and source correspondence

Let μ be a finite positive measure on $\mathbb{R}$, $a \le b$, and assume μ-almost every x lies in $[a,b]$. Fix $z \in \mathbb{C} \setminus \{t : t \in [a,b]\}$, with the canonical embedding $\mathbb{R} \hookrightarrow \mathbb{C}$ understood. Put

$$M = \mu(\mathbb{R}), \qquad F_\mu(t) = \mu((-\infty, t]), \qquad r(t) = \frac{1}{z-t}.$$

Here finite real masses are used; this is not an extended-real multiplication. The directly inspected ProveIt theorem establishes

$$\int_{\mathbb{R}} \frac{d\mu(x)}{z-x} = \frac{M}{z-b} - \int_a^b \frac{F_\mu(t)}{(z-t)^2}\, dt. \tag{8.1}$$

Its Lean implementation uses real scalar multiplication on complex values, `Measure.real`, and oriented interval integrals [3]. Those representation choices are part of the source correspondence, even when a mathematical renderer uses conventional notation.

8.2 The complete mathematical argument

Define $k(t) = (z-t)^{-2}$. Domain exclusion gives $z - t \neq 0$ on $[a,b]$. Thus r and k are continuous there, and

$$r'(t) = k(t), \qquad \int_x^b k(t)\, dt = r(b) - r(x) \quad (x \in [a,b]). \tag{8.2}$$

Continuity on the compact interval and finiteness of μ, together with its almost-everywhere support, imply μ-integrability of r.

The closed-CDF layer-cake identity is

$$\int_a^b F_\mu(t)\, k(t)\, dt = \int_{\mathbb{R}} \left(\int_x^b k(t)\, dt \right) d\mu(x). \tag{8.3}$$

One way to see it is to write $F_\mu(t) = \int \mathbf{1}_{\{x \le t\}}\, d\mu(x)$ and exchange the integrals. On the supported interval the absolute value is bounded by a constant times $M(b-a)$, so the interchange is justified. For supported x, the inner integral runs from x to b. Endpoint choices affect a Lebesgue-null set in the t-integral and do not remove atoms of μ.

Substitute (8.2) into the right side of (8.3) almost everywhere:

$$\int_a^b F_\mu(t) k(t)\, dt = M r(b) - \int_{\mathbb{R}} r(x)\, d\mu(x).$$

Rearranging gives (8.1). The degenerate interval $a = b$ is also covered: its interval integral is zero and the measure is supported at that point. The source proof realizes this argument through continuity, interval-integrability, support restriction, the fundamental theorem, and integral-congruence lemmas [3].

8.3 Which obligations should be implicit?

The plan's domain layer proves

$$z \notin \operatorname{complexify}[a, b] \Longrightarrow \forall t \in [a, b],\ z - t \neq 0.$$

The regularity layer consumes that proposition to derive continuity and the derivative formula. The integration layer uses the fixed measure, finiteness, and support to derive integrability and almost-everywhere replacement. The author supplies the layer-cake method and primitive identity; the elaborator supplies well-typed applications of their standard premises.

These are different interfaces, not one universal "regular" flag. In particular, an μ-integrability fact is not a ν-integrability fact, and a derivative formula valid on $[a, b]$ must be restricted correctly to $[x, b]$. Sorrel's plan must retain the interval-membership evidence needed for each subinterval application.

A compact view can expose the following dependency summary without listing all coercion lemmas:

Requested operation	Sufficient evidence consumed
Differentiate r	Nonzero denominator at the current point.
Use the primitive on $[x, b]$	$x \in [a, b]$; derivative on that subinterval; interval integrability.
Replace under μ	Almost-everywhere support and the pointwise primitive for supported points.
Integrate $r(b) - r(x)$	Finiteness of μ and μ-integrability of r.
Normalize mass to one	A separate probability-measure hypothesis.

8.4 Positive and negative neighbors

For a point mass $\mu = \delta_c$, $c \in [a, b]$, the left side of (8.1) is $(z - c)^{-1}$, while $F_\mu(t) = \mathbf{1}_{\{c \leq t\}}$. The right side is

$$(z - b)^{-1} - \int_c^b (z - t)^{-2}\, dt = (z - c)^{-1}.$$

This is an important positive fixture: no atomlessness premise is needed. Requiring one would weaken the source theorem unnecessarily.

Removing domain exclusion should expose the nonzero-denominator requirements of this route. It must not produce a universal claim that the identity is false for all such inputs. For example, the zero measure can make a formula trivial despite an unsuitable primitive route. Similarly, replacing finite mass by unit mass without a probability hypothesis is a genuine change of statement; a Dirac measure of mass two detects it immediately.

Almost-everywhere equality must not be promoted to endpoint equality. Under Lebesgue measure, changing a function at one point preserves its integral but can change its value at that endpoint. A consumer registry can use $f =_{\mu\text{-a.e.}} g$ to replace an integrand, while refusing point application unless additional evidence is supplied.

8.5 An explicit consumer registration for almost-everywhere rows

A registered integral consumer has schematic type

$$(\forall^{\mu\text{-a.e.}} x,\ f(x) = g(x)) \longrightarrow \int f\, d\mu = \int g\, d\mu.$$

Other methods, such as distributing a difference into two integrals, have additional integrability premises. The registry retains these exact theorem interfaces rather than assuming that all integral

operations share one guard. For a family of almost-everywhere facts, a countable intersection may be available under the appropriate theorem; an arbitrary uncountable intersection is not inferred.

This answers a round-four question concretely: almost-everywhere information enters as a relation with the measure in its anchor, and consumers are ordinary Lean theorems with checked binder roles. It is not an ambient change from pointwise mathematics to a quotient where all consumers become valid.

9 Arithmetic and dependent construction controls

9.1 Exact Frey quotients without a contradiction shortcut

The Frey package contains nonzero integers a, b, c, a prime exponent $p \geq 5$, the Fermat equation, a gcd condition, and congruences. Its integral and rational curve definitions both display

$$a_2 = \frac{b^p - 1 - a^p}{4}, \qquad a_4 = \frac{-a^p b^p}{16},$$

but division has different semantics in the two coefficient domains [5]. The source bundle illustrates Lean's existing property-carrying capability; Sorrel's task is extracting the premises that each use needs.

Following the corrected round-three benchmark, use instead the satisfiable arithmetic context

$$a, b \in \mathbb{Z}, \quad p \in \mathbb{N}, \quad p \geq 4, \quad p \text{ odd}, \quad a \equiv 3 \pmod 4, \quad 2 \mid b. \tag{9.1}$$

Since $b = 2u$, $16 \mid b^p$, and hence $16 \mid -a^p b^p$. Also $b^p \equiv 0 \pmod 4$ and $a^p \equiv (-1)^p = -1 \pmod 4$, so $4 \mid b^p - 1 - a^p$. Only evenness and the exponent bound are needed for a_4; the other assumptions belong to the chosen a_2 route. This is a reuse of an already analyzed benchmark, not a newly discovered number-theoretic result [1, 2].

Proposition 9.1 (Balance before inversion)**.** *If $d, q, n \in \mathbb{Z}$ satisfy $dq = n$, every unital ring homomorphism $j : \mathbb{Z} \to R$ gives $j(d)j(q) = j(n)$. When R is a field and $j(d) \neq 0$, it follows that $j(q) = j(n)/j(d)$.*

Proof. Apply j to the balance equation and use preservation of multiplication. In the field case multiply by the inverse of $j(d)$, or apply the corresponding cancellation theorem. □

The type distinction is essential. Nonzero integers may map to zero in positive characteristic. A regular element can be cancelled but need not be a unit; 2 in $\mathbb{Z}$ is the elementary example. Sorrel records exact quotient construction, balance transport, and inversion as different methods.

For $(a, b, p) = (3, 2, 5)$, the coefficients are -53 and -486. The executed model checks these values and the four single-premise neighbors $(3, 2, 4)$, $(3, 3, 5)$, $(3, 2, 3)$, and $(1, 2, 5)$. They respectively detect removal of oddness, evenness, the exponent lower bound for a_4, and the residue condition for a_2. These finite checks are not a replacement for the universal proof; they are useful tests of the intended benchmark.

The full counterexample package is eventually contradicted in an FLT proof. That is legitimate mathematics. But testing an arithmetic interface only in such a context may allow an irrelevant contradiction proof to pass. Sorrel's strict arithmetic plan uses a restricted helper context with a positive fixture. It does not outlaw proof by contradiction throughout the language.

9.2 Sharp regularity is a stronger contract than a bound

ProveIt's regularity file uses

$$F'(x) = 2\,\mathrm{up}(2x-1), \qquad 0 \le \mathrm{up} \le 1,$$

to establish a global Lipschitz constant two. It then uses $F'(1/2) = 2$ to show that no smaller constant works [4]. A suitable compact plan is:

```
have abs(F'(x)) <= 2 for all x by differential_law and range_bounds
have Lipschitz(F,2) by mean_value
have F'(1/2) = 2 by differential_law and normalization
have 2 is the least Lipschitz constant of F
  by derivative_lower_bound and the established upper bound
```

The third line supplies additional evidence, not decorative explanation. From F being K-Lipschitz, each difference quotient has magnitude at most K; its limit gives $|F'(1/2)| \le K$, so $2 \le K$. Without such a lower-bound argument, the leastness obligation stays open. Attainment at a point is one sufficient route, not the only possible one.

The source already contains short consequences obtained from the Lipschitz statement. Sorrel should not replace those with a larger ritual. This example is a regression test for result strength and a baseline for whether a new property interface improves on an existing theorem application.

9.3 A representing algebra needs a dependent package

The inspected FLT solution assumes a finite index type I, a commutative ring A, and commutative A-algebras H_i that are each finite and free as A-modules. It constructs an A-algebra W, also finite and free, with equivalences

$$e_D : \mathrm{AlgHom}_A(W, D) \simeq \prod_{i \in I} \mathrm{AlgHom}_A(H_i, D),$$

natural in the target algebra D [6].

Its proof uses induction on a finite type: the empty case uses A; the successor case tensors the previous representing algebra with the new factor; the equivalence case transports the indexing family. The successor assembles several equivalences and then proves naturality componentwise. A Sorrel plan can make that construction explicit while delegating routine adapter composition:

```
construct a representing algebra by finite-family induction
  empty: use A with its canonical A-algebra structure
  extension: tensor the previous witness with the new factor
  equivalence: transport the family along the given equivalence
retain the chosen algebra structures and the induced hom equivalences
establish naturality by composition of the construction's naturality laws
```

Each step returns data and laws about that data. The finite/free guarantees must remain attached to the selected algebra and module structures. An unordered union of evidence from two different constructed witnesses does not make one package.

The factor hypotheses are necessary for the advertised general guarantee: for a singleton family, a natural representing object is isomorphic to its single factor; $\mathbb{Q}[X]$ is not finite-dimensional over $\mathbb{Q}$. This is the premise omission corrected in Moraine's exposition by the syntheses, not a bug in the original Lean theorem [1, 2, 6].

Finally, the source conclusion is existential. It does not promise an executable algorithm producing the package from a proof of abstract finiteness. Sorrel must distinguish choosing a witness inside

a proof, declaring a noncomputable construction under a choice policy, and exporting executable data. Merely replacing `exists` with `construct` cannot strengthen that guarantee.

10 Leant as a plan-completion service

10.1 Preserve the existing acceptance boundary

Leant's inspected `Verified` wrapper explicitly records acceptance by a supplied verification callback. Its nominal role and opaque constructor protect association with the candidate; they do not turn that receipt into a kernel proof or behavioral theorem. The source says that stronger authority must be retained and replayed separately [7].

The interface groups rendered variants, tries them in order, and records the first accepted representative. Its keyed version deduplicates accepted renderings without transferring semantic authority to a different occurrence. Sorrel should retain this careful candidate identity rather than replacing it with a vague "AI verified" status [7].

The revised syntheses report a later Leant path that checks a supplied exact behavioral proposition about a candidate and treats a successful negation check differently from failure of a positive attempt. Sorrel's integration must accommodate that stronger baseline while still retaining the proof and source correspondence needed by its own publication boundary [1, 2].

10.2 Send a typed residual request, not prose

A plan-completion request contains the frozen target, the local dependent context, available premises, permitted providers, and the authorized kind of hole. It may ask for a proof of a guard, construction of an adapter, or a function satisfying a behavior specification. A data hole is accompanied by its specification; the service is not free to alter it.

An adapter is a particularly plausible synthesis target. For example, given an invariant predicate and a map preserving it, synthesize the map on the subtype, then prove its behavior from the original law. Another target composes two contracts using Proposition 4.1. These exercises genuinely require term assembly; a successful lookup of one existing theorem should be reported separately.

A practical implementation can treat difficult dependent propositions as opaque proof inputs to a structural synthesis fragment, then reconstruct and check the complete term in Lean. It should not claim that Leant's structural search automatically decides arbitrary dependent behavior. Unsupported translations must remain explicit, consistent with Leant's documented fragment boundaries [8].

10.3 Behavioral pruning before completion

Suppose a partial program s denotes a set $\mathcal{C}(s)$ of authorized completions. Rejecting one completion $c \in \mathcal{C}(s)$ excludes that candidate only. Pruning the whole sketch requires a necessary-condition theorem covering every permitted completion, or a heuristic status that does not claim logical impossibility.

Residual interfaces offer a disciplined source of such constraints. A consumer may demand a universally length-preserving operation, an invariant-preserving map, or a relational pair of outputs. The synthesis engine can propagate these specifications into holes using checked implication rules. If a propagation rule only supplies a sufficient construction strategy, failure of that strategy cannot refute the sketch.

For sorting, the frozen specification should say

$$\forall xs,\ \mathsf{Sorted}(f(xs)) \wedge \mathsf{Perm}(f(xs), xs),$$

with the order relation fixed. Stability is a separate further property if requested. Length preservation is a useful filter, not a substitute for the specification. The exact selected function term must occur in the theorem accepted at the boundary.

10.4 Positive and negative abstraction contracts differ

Let an admissible input $x : A$ have an observation $o(x) : U$, and let a model $\widehat{f}$ commute with the concrete operation through a proved semantic bridge. A universal model theorem over all $u : U$, together with a forward transfer into the desired concrete specification, can prove every admissible concrete case. Extra unrealizable abstract inputs do not invalidate such a positive proof.

A negative abstract witness is different. To refute the concrete specification, there must be an admissible x realizing the witness and a theorem that a true concrete specification would imply the tested abstract condition. This is not implied by positive transfer alone. Surjectivity of the observation is sufficient but stronger than necessary; realization of the particular witness suffices.

For list lengths, the possible observations depend on the element type. At `List Empty`, only zero is realized. A constant-empty function and the identity therefore have the same concrete length behavior, even though their unrestricted affine models 0 and n differ. With a supplied element, arbitrary finite lengths are realizable. Two individually realizable observations also need not be jointly realizable from the same input. Sorrel retains the entire configuration and its admissibility conditions [1, 2].

The interface consequently distinguishes: a checked source proof; a checked source refutation; a model-relative counterexample awaiting realization; an unsupported fragment; and an exhausted search. Only the first two establish a logical verdict about the original candidate and specification.

10.5 Replay and retention

A successful suggestion is accepted by replaying the exact selected source in the original proof state, accounting for all generated goals. The returned proof, fixed target, provider identities, environment pin, and axiom policy are retained. A later pretty-printer rewrite is a new candidate unless it is rechecked or connected by an adequate typed bridge.

A service can use language models to rank plans, propose intermediate lemmas, or suggest a construction. Those proposals are untrusted inputs to the same acceptance boundary. A fluent explanation is not behavioral evidence, and a model's choice of a more convenient target is not a proof of the author's request.

11 Checked computation as an operation with a contract

11.1 One request, several permissible execution lanes

A CAS-style request in Sorrel has an input, an output type, and a fixed relation connecting the output to the original mathematical object:

$$\mathsf{Request}(x) := \Sigma y : O(x),\ \mathsf{Spec}(x, y).$$

For executable production this is a data package (implemented with an ordinary structure or subtype when the second field is propositional); a theorem asserting its existence is a different

artifact. Suitable requests include an exact polynomial identity, a conditional rational-function simplification, an isolating interval for a specified root, a finite jet of a selected formal solution, or an enclosure with a stated error. These are separate relations, not success levels on one scale.

Three lanes can satisfy the request. A verified algorithm has a correctness theorem and is evaluated through an accepted route. An untrusted searcher returns a certificate checked by a verified checker. An existing proof-producing Lean tactic returns a proof directly. The source contract is the same; checking cost and retained evidence differ. The two syntheses already insist on this separation [1, 2].

11.2 A shared reification interface, not a universal normalizer

A reusable reifier can target a ring-expression grammar with literals, variables, addition, negation, and multiplication. For each original Lean expression e, it returns a syntax tree q, an ordered environment of opaque atoms v, and a bridge

$$\beta_e : \quad e = [\![q]\!]_v.$$

Opaque atoms are exact retained terms, not arbitrary printed names. Different values may share a symbol only if the actual correspondence is justified. The grammar and denotation specify the coefficient domain and chosen ring structure.

A polynomial identity checker may then work with normalized coefficients. An affine inequality checker needs additional ordered semantics and a theorem about nonnegative combinations. A length abstraction needs a bridge from list operations to natural-number expressions, not merely a ring reifier. Sharing expression syntax does not make these semantic theorems identical.

The original-target chain is explicit:

$$e \overset{\beta_e}{=} [\![q]\!]_v \overset{\text{checked certificate}}{=} [\![q']\!]_v \overset{\beta_{e'}^{-1}}{=} e'.$$

A correct equality about a different list of atoms proves the wrong theorem; the reification bridges are therefore part of acceptance, not optional metadata.

11.3 Soundness, completeness, and execution are separate theorems

A typical positive theorem is

$$\mathrm{check}(x, c) = \mathsf{true} \Longrightarrow \mathsf{Spec}(x, \mathsf{out}(c)).$$

It proves every accepted result without requiring the checker to accept all true specifications. Decision completeness is a separate converse, restricted to a specified grammar and admissible domain. Negative certification needs its own theorem, not the failure of this implication's antecedent.

There is also an execution obligation. A theorem about the checker as a Lean function does not certify arbitrary bytes reported by a native executable. The strict publication profile uses a retained proof or a checker computation replayed through an accepted kernel-level route. Accelerated native evaluation must be documented under its actual trust and axiom policy for the selected toolchain. This paper does not claim a performance threshold at which that tradeoff becomes worthwhile.

11.4 Precision is an index with algebraic laws

A jet can be specified by agreement of coefficients below order N, or congruence modulo X^N. Differentiation reduces a guaranteed order by one: for $N \geq 1$, agreement modulo X^N implies derivative agreement modulo X^{N-1}. Multiplication, composition, and inversion have their own premises and precision rules. A residual equation for a formal root also needs a branch or constant-term condition and an applicable uniqueness theorem.

These requirements fit the same proof-plan interface. The consumer states its needed precision; registered theorems propagate sufficient requirements back to its inputs. An approximation is never silently presented as equality of whole functions, and a polynomial root certificate is never silently promoted to completeness of the solution set. To claim complete solutions, a result needs both soundness of its listed solutions and coverage of all admissible solutions.

12 Implementation architecture and its first useful slice

12.1 Components with explicit trust boundaries

The proposed Lean package has six components. A source elaborator creates the frozen header and typed plan nodes. A theorem registry validates signatures and binder roles. A local property index points to accepted expressions in the Lean context. A grounding and support service schedules eligible proof routes. Provider adapters call ordinary tactics, Leant, or certificate generators. Finally, an evidence compiler and renderer retain the accepted terms and produce the human-facing explanation.

Only Lean's proof checking determines theorem acceptance. The frontend and renderer still bear separate responsibilities for source fidelity; their bugs can make a correct theorem look like a different authored claim. Method-policy checking adds another obligation and is not folded into a misleading notion that every trusted function has the same purpose.

The first implementation should generate explicit Lean proof applications and share intermediate results through local declarations. It need not maintain a second independently authoritative context or a global database of every mathematical fact. Provenance and source positions index existing evidence. The publication artifact includes a reproducible Lean file, not only an editor screenshot or a process-local receipt.

12.2 A concrete incremental delivery plan

Slice A: one proof-implicit use site. Wrap exact quotient transport with fixed data arguments and explicit divisibility/nonzero premises. Register only the few arithmetic bridge theorems needed by the nonvacuous fixture. Accept an existing Lean proof as a way to fill every obligation. The interface must export the same target under an explicit support-restricted context.

Slice B: two sufficient routes. Add the finite support engine, using Lean expressions rather than the companion's atom names. Show the direct and parity-based divisibility routes, remove oddness, and display the resulting alternative frontier. Require an actual proof for the chosen repair.

Slice C: analysis transfer. Add the Cauchy example's consumers, including almost-everywhere replacement. Compare with the same helpers and `fun_prop` available in ordinary Lean. Verify that mass two, a point atom, and a changed measure receive the intended meanings.

Slice D: Leant and dependent construction. Complete a genuine adapter hole, then inspect the induced-subtype or tensor-family example. Record when the grounding fragment refuses

a data-dependent obligation. That refusal is a useful boundary, not a reason to approximate dependent types unsafely.

Input grammar beyond the proof-implicit binder is optional until these slices show a benefit. The full narrative surface can first exist as a read-only renderer over accepted Lean plans. A renderer-only or library-only outcome is an acceptable result of the experiment.

12.3 Lexical structures: an experiment, not a magic preamble eraser

A lexical structure context records which ring, topology, scalar action, and measure each expression uses. It helps prevent ambiguity and provides explicit adapters when an API requires local typeclass instances. It does not imply that every generated FLT preamble disappears. Some preamble choices control search or resolve overlapping instances, and a new interface can merely move that cost elsewhere.

The reviewed tensor solution already relies on carefully scoped structures and introduces finite/free instances from its inductive witness. Sorrel should preserve their dependency on that witness. A separate controlled experiment can compare preamble size, elaboration cost, and repair cost with and without a lexical context mechanism. This paper does not report that experiment.

13 The executable companion: what it establishes

13.1 A narrow frontend that actually runs

The archive contains `sorrel_model.py`, a dependency-free Python program implementing the finite calculus of Section 6. Its small `.sorrel` input format declares a lexical context, proposition names, assumption slots, currently available slots, a finite implication table, and a goal. This is not the mathematical language illustrated in Section 3.

The quotient input is:

```
context ExactQuotient
atom EvenB LargeP OddP ResidueA Direct4 Dvd4 Dvd16 Both
assumption EvenB LargeP OddP ResidueA Direct4
available EvenB LargeP ResidueA
rule power16: EvenB & LargeP -> Dvd16
rule power4: EvenB & LargeP & OddP & ResidueA -> Dvd4
rule direct: Direct4 -> Dvd4
rule pair: Dvd4 & Dvd16 -> Both
goal Both
```

The output gives two minimal supports, the residual alternatives {Direct4} and {OddP}, and the status `open`. The model does not claim to have proved the arithmetic rules. They are abstract interfaces assumed by this experiment. The separately executed integer fixtures test selected positive and negative values.

For each retained support, the solver retains a finite proof tree. A separate checker validates the target, rule identifier, ordered premise list, arity, context, and exact assumption leaves. It does not consult the solver's labels. The exporter turns one selected tree into an ordinary Lean implication whose proposition meanings and used rules remain explicit parameters.

13.2 Executed results

The test run used Python 3.13.5 and deterministic seed 20260906. The exact result file is `test_results.json`.

Executed quantity	Count
Named regression tests	31
Generated finite rule systems	80
Assumption subsets exhaustively checked across those systems	1,280
Subset/goal-atom reachability comparisons	7,680
Retained certificates checked in generated systems	517
Boolean valuations enumerated	5,120
Support soundness checks inside valuations satisfying the rule table	4,550

Each generated system has six proposition atoms, four possible assumption slots, and ten randomly generated rules of arity zero through three. For every one of its 16 assumption subsets, an independently implemented Boolean Horn closure is compared with the support engine's predictions for all six atoms. All 64 Boolean valuations are also enumerated; valuations satisfying the rule implications are used to validate the returned supports. These are finite exhaustive checks of generated cases, not formal proofs of the Python implementation. The general arguments are the written theorems in Section 6.

The named tests include alternative supports, residual minimization, closing and reopening a route, an unseeded cycle, a seeded finite derivation, deletion of its seed, zero-premise rules, policy filtering, rule-order invariance, altered targets, sibling contexts, changed rule tables, malformed certificates, parser errors, exhausted budgets, and the four Frey mutations. A small witness interface also rejects positive list length when no element witness is admitted; it is not a Lean proof about `Empty`.

13.3 What was not executed

No Lean executable was available in this environment. Consequently `SelectedPlan.lean` is labeled *generated but uncompiled*; it is not counted among checked theorems. The archive does not claim a real Lean metavariable freeze, typeclass registry, reifier, theorem transport, Leant call, Cauchy theorem compilation, or benchmark of the repositories.

The miniature parser operates on named propositional atoms, not on Lean mathematical terms. Its context identifier prevents an elementary scope mixup, but does not validate equivalence of rebuilt Lean contexts. Its proof checker checks conditional implication syntax, not the truth of the user-declared rules. The explicit budget limits rule-combination attempts, not all possible memory consumption or external provider time. These limits are part of the artifact's meaning.

The model nevertheless improves on a nonexecuted pseudocode specification: it fixes input syntax, distinguishes open and supported results, retains alternative supports, exercises corruption and deletion behavior, and exports one concrete conditional proof expression. Those are the pieces proposed for porting to the first Lean slice.

14 Evaluation: separate the property layer from its helpers

14.1 Matched arms

The crucial comparison is not Sorrel against an unassisted author typing raw Lean from scratch. Use matched arms with shared libraries and the same mathematical interfaces:

Arm	Capability
L0	Ordinary Lean and its applicable existing automation.
L1	L0 plus the newly packaged mathematical helper theorems and adapters.
L2	L1 plus the same Leant, search, and certificate-producing services.
L3	L2 plus Sorrel's proof-implicit interface, property index, and residual plans.
L4	L3 plus the proposed narrative source syntax.

A read-only renderer over accepted L2 proofs is an additional useful control. L3 versus L2 isolates the property/plan layer; L4 versus L3 tests whether new input syntax is justified. Registration and helper-writing time must be charged to the arms that need it, rather than treated as free preparatory work.

14.2 Tasks and intervention logs

The source files discussed here are development material, not held-out tasks. They establish acceptance tests and guide implementation. A separate assessor should select transfer problems after the interface and registry policy are frozen. Suitable families include local analytic reasoning, invariant-subtype adapters, and dependent naturality constructions not used while designing the rules.

Record author interventions, elapsed authoring and repair time, successful single-lemma lookup versus genuine composition, rule instances generated and used, support-label sizes, proof-node counts, elaboration/checking time, and cold versus warm rebuild cost. Retain failures as well as successes. Report time spent registering a special theorem for one task separately from reused infrastructure.

An edit study is particularly appropriate for Sorrel. Delete or weaken a premise, change a measure, rename a binder, change an index, or restrict the allowed method. Measure whether the fixed target remains unchanged, which plans survive, which residual obligations appear, and how much author effort is required to complete a valid repair.

14.3 Reading comprehension and semantic mutations

Before revealing an expanded proof, ask readers to reconstruct the compact claim: objects, domain, quantifiers, measure, branch, exactness, and open conditions. Include polished invalid neighbors. Then reveal the expansion and measure whether it corrects misunderstanding. A compact source that reliably makes readers believe a stronger theorem is not successful compression.

The initial acceptance suite should include at least the following paired situations. Each negative case needs a nearby admissible positive fixture; some negative outcomes are rejected promotions, not false mathematical statements.

Boundary	Positive fixture	Required response to mutation
Conditional evidence	All leaves of a supported plan are supplied.	Removing one leaf reopens the plan; no silent extra premise.
Locality	Equality on a neighborhood of the evaluation point.	Equality only at the point does not justify derivative transport.
Almost everywhere	μ-a.e. equality used by a μ-integral consumer.	Changing μ or requesting a point value requires new evidence.
Exact division	The satisfiable fixture $(3, 2, 5)$.	The four removed-premise fixtures fail the corresponding divisibility claim.

Boundary	Positive fixture	Required response to mutation
Result strength	Lipschitz upper bound plus a lower-bound argument.	An upper bound alone does not establish leastness.
Abstract refutation	A concrete input realizes the failing observation.	Positive length with no element witness remains unrealized.
Structure choice	Finite/free factor algebras and retained selected structures.	Finite indexing alone does not supply finite/free factors.
Method fidelity	A permitted rule tree with restricted helper premises.	An unrelated contradiction proof does not satisfy that method policy.
Source integrity	Every displayed assertion has accepted evidence.	An unused false assertion prevents complete publication.
Rebuild integrity	Evidence is rechecked in the destination context.	Old local identifiers or matching printed names alone are insufficient.

14.4 Stopping rules

Set quantitative thresholds after a pilot estimates variability, and before the main comparison. This paper invents no productivity percentage. The separate-language experiment should stop or shrink when L3's gains vanish against L2, its explanations increase semantic errors, or its registration and repair costs exceed the work it saves.

A useful outcome may be a library of mathematical contracts, an evidence-aware editor, a Leant adapter, or a read-only renderer. Retaining those pieces while dropping the narrative language would follow the evidence rather than defeat the proposal.

15 Answers to the two reports' fourth-iteration questions

The two question lists overlap but emphasize different implementation risks. The following crosswalk states Sorrel's decision and the evidence still needed, rather than treating a design answer as an experiment already completed [1, 2].

Question cluster	Sorrel decision	Current status
First use site and package	Proof-implicit exact-quotient transport; two alternative divisibility routes; restricted arithmetic premises.	Finite symbolic slice executed; Lean integration proposed.
Finite rule generation	Explicit finite term bank, schema whitelist, bounded constructors and instantiations; exactness only relative to generated rules.	Algorithm and bounds specified; Lean grounding not built.
Data inference and structures	Freeze meaning-bearing data; delegate witnesses only through a fixed specification; retain dependent structure packages.	Design; source examples inspected.
Required proof methods	Filter rules before minimization; check typed plan nodes and support-restricted helper contexts separately from kernel truth.	Rule-filter test executed; full policy compiler proposed.
Behavioral bridge and completeness	Keep positive soundness, exact decision, and negative realization as separate provider contracts.	Interface analysis; no new Lean behavioral checker.

Question cluster	Sorrel decision	Current status
Negative Leant outcomes	Refutation needs source proof or realized counterexample with transfer; one failed completion cannot refute a sketch.	Source review and design.
Nonalgebraic transfer and a.e. consumers	Cauchy–CDF example, measure-anchored relation, explicit integral-congruence consumer.	Mathematical reconstruction; not a held-out task.
Rewriting, indices, and lifetime	Keep original anchors; insert checked bridges; rebuild per declaration before attempting cache transport.	Elementary context rejection tested; dependent transport proposed.
Reifier and checker reuse	Shared ring syntax with per-expression denotation proofs; distinct ordered and behavioral semantic layers.	Specification only.
Actual cost and rule usage	Instrument labels, generation, lookup/composition, proof growth, and cold/warm elaboration in matched arms.	Finite operation counts only; no Lean benchmark.
Lexical preambles	Measure instance-context changes rather than assume a preamble disappears.	No preamble-compression result claimed.
Reader recovery and number of implementations	Blind reading and edit studies; independently implement/check the same small interface before expanding syntax.	Evaluation protocol; no participants tested.
Evidence migration and stop conditions	Recheck exact target/dependencies at destination; keep library/services if the language adds no benefit.	Acceptance policy and proposed stopping rule.

16 Risks, priorities, and conclusion

16.1 The largest risks

The first risk is search explosion. Minimal support sets make explanations more informative but can become exponentially numerous. The initial engine therefore needs small profiles, explicit completeness status, and demand-driven use. Top-ranked partial explanations can be useful, but must not be advertised as the complete set of repairs.

The second risk is an expensive second elaborator. Duplicating instance search, function-property automation, and rewriting can erase the author's savings. Sorrel should orchestrate existing proof-producing tools and retain only additional information that is demonstrably useful for source meaning, residual interfaces, or repair.

The third risk is false precision in explanations. A minimal support in a fragment can still be redundant in mathematics, and a strict method grammar can reject perfectly reasonable alternative proofs. The interface must state its scope and permit an explicit author choice to change the proof route.

The fourth risk is representational drift: changed measures, coefficient structures, universes, witnesses, and dependent indices. An uncompiled finite prototype does not solve it. Expression-level acceptance checks and rebuild tests are therefore higher priority than more surface vocabulary.

16.2 What to implement next

The next engineering artifact should be the exact-quotient use site in Lean, with the source target frozen, one property-implicit binder, the two support routes, explicit residual display, and a retained kernel-checked proof. The independent evaluator should receive the paired mutations before the frontend is expanded. The Cauchy example should follow as an analysis transfer test within the development suite, not as a claimed unseen benchmark.

Only after those experiments should Sorrel's narrative input be enlarged or a native refinement former be attempted. The mathematical property types are important; their successful use in an actual proof state is the decisive unit.

16.3 Conclusion

Sorrel proposes a type system for *arguments in progress*. An object's proved properties determine which operations can consume it. A function's behavior is a theorem about its actual evaluations. A mathematical plan states its target and exports a typed interface of the evidence still needed. Alternative sufficient interfaces can be retained and inspected without pretending to compute globally weakest assumptions.

This changes what authors must write while preserving the role of Lean's kernel. The author selects the mathematical route and meaningful construction choices; Lean checks the completed evidence; Leant and certified computation fill permitted gaps; the interface explains remaining conditions without changing the proposition.

The contribution is deliberately narrower than a finished language. The finite calculus has written proofs and an executed reference model. The corpus reconstructions identify what a real frontend must preserve. The measurement protocol can reject the separate-language hypothesis while keeping useful libraries and tools. That is a stronger fourth-round position than another promise that better syntax alone will make formal mathematics concise.

A Reproducing the companion and the article

The archive is self-contained for rebuilding the PDF with a conventional pdfLaTeX installation. The article embeds its bibliography and requires no external figures or font files. The companion requires Python 3.10 or newer and no third-party packages.

```
python sorrel_model.py --test --json test_results.json
python sorrel_model.py examples/quotient.sorrel \
  --json examples/quotient_result.json \
  --export-lean SelectedPlan.lean
python sorrel_model.py examples/cauchy.sorrel \
  --json examples/cauchy_result.json
pdflatex -interaction=nonstopmode -halt-on-error sorrel.tex
pdflatex -interaction=nonstopmode -halt-on-error sorrel.tex
pdflatex -interaction=nonstopmode -halt-on-error sorrel.tex
```

A budgeted model run is requested with `-budget N`. An incomplete support inventory can still contain a valid conditional derivation; the output exposes both statuses. A malformed input exits with an error rather than silently inventing a proposition or rule.

The generated Lean specimen uses `Init` and states a propositional implication with its used rule interfaces as hypotheses. Running `lean SelectedPlan.lean` is an intended additional check on a machine with Lean installed. It was not performed here, and compiling it would still not prove the mathematical meaning of the model's rule names.

B A compact interface schema

The following is a design schema, not a claimed implementation:

```
FrozenRequest
  target        : elaborated Lean expression
  context       : ordered local declaration telescope
  universes     : fixed level assignments
  protectedData : fixed structures, domains, witnesses, and relations
  allowedHoles  : proof goals or explicitly specified constructions

PlanNode
  originalRequest : FrozenRequest
  operation       : checked theorem interface or explicit constructor
  dataArguments   : typed expressions
  premises        : ordered premise nodes
  residual        : well-formed dependent telescope
  result          : expression checked under the residual telescope
  methodPolicy    : optional accepted-plan grammar and dependency policy

AcceptedArtifact
  statement      : exact checked target
  evidence       : retained Lean term or replayable checked certificate
  sourceMap      : every displayed assertion to its evidence node
  dependencies   : environment and actual trust/axiom policy
  planRecord     : required method evidence, separately validated
```

Publication checks all display nodes and confirms that each frozen request's residual telescope is filled. A user may instead publish an explicitly conditional theorem; its statement then contains the conditions. A conditional artifact and an unconditional theorem must not share a status that

suppresses this difference.

C Provenance and bibliographic notes

Repository citations below link to the pinned files, not to mutable default branches. The record `sources.json` includes complete commit and blob identities for the main inspected files. Line ranges stated in that record are source-file ranges, not PDF page numbers. No upstream source archive is redistributed. The inspection date for live official documentation is 6 September 2026.

The reports' attributions to earlier proposals are retained as attributions: Trellis's finite qualifier closure motivates making the grounding boundary explicit; Fiber's and Obsidian's nonvacuity warnings motivate the arithmetic fixture; Moraine's behavioral model and Karst's empty-type example motivate separate realization contracts. Sorrel's finite support sets also have a clear prior-art relationship to the ATMS label of minimal environments. Unlike a full default-reasoning ATMS, this model neither assumes consistency checking nor removes mathematically contradictory contexts automatically. It computes positive conditional derivability from a fixed implication table.

References

[1] Codex. *From Properties to Proof Obligations: Round 3 Synthesis*. Brainstorm, revised round-three synthesis, 6 September 2026. Commit `bf3333905995060870f8585e297ffc16f3fe7e86`. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex.

[2] Claude (Anthropic). *Nine Refinements: Property-Aware Typing in a Mathematician's Proof Language over Lean*. Brainstorm, revised round-three review, 6 September 2026. Same Brainstorm commit as above. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex.

[3] ProveIt contributors. *Cauchy transforms through compactly supported CDFs*. `Analysis/FabiusFunction/Lean/FabiusFunction/CauchyCDF.lean`, commit `24ce8bd743eaab64a91ce90725ea00f498d319d2`. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/CauchyCDF.lean.

[4] ProveIt contributors. *Sharp Lipschitz regularity of the Fabius and Rvachev functions*. `Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean`, same ProveIt commit. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean.

[5] Fermat repository contributors. `Definitions/Def_FLTPrelim_FreyPackage.lean`. Source credits Kevin Buzzard, Ruben Van de Velde, and Pietro Monticone; ported from the Imperial College London FLT formalization. Commit `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_FLTPrelim_FreyPackage.lean.

[6] Fermat repository contributors. `P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean`. Same Fermat repository commit; finite/free factor hypotheses in the `solution` theorem, followed by the naturality conclusion and proof. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean.

[7] Vladimir Reshetnikov and Leant contributors. `src/Leant/Synth/Verification.hs`. Commit `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs.

[8] Vladimir Reshetnikov and Leant contributors. *Leant: interactive Lean 4 helper*, repository README. Public description inspected 6 September 2026; implementation claims in this article are bounded by the specific source review noted above. https://github.com/VladimirReshetnikov/Leant.

[9] Mathlib contributors. *Mathlib.Tactic.FunProp*, official module documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.

[10] Lean contributors. *The Lean Language Reference: Subtypes*. https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/.

[11] Lean contributors. *The Lean Language Reference: Axioms*; and *Theorem Proving in Lean 4: Propositions and Proofs*. https://lean-lang.org/doc/reference/latest/Axioms/. https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/.

[12] Lean contributors. *Lean 4.33.0 release notes*, 10 August 2026. In particular, automation, incremental elaboration, and the renaming of experimental `mvcgen'` to `vcgen`, leaving `mvcgen` unchanged. https://lean-lang.org/doc/reference/latest/releases/v4.33.0/.

[13] Johan de Kleer. *Problem Solving with the ATMS*. Artificial Intelligence 28 (1986), 197–224. The introductory description of minimal-environment labels and implication justifications is especially relevant. https://dekleer.org/Publications/Problem%20Solving%20with%20the%20ATMS.pdf.

[14] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. Proceedings of PODS 2007, 31–40. https://doi.org/10.1145/1265530.1265535.

[15] William Lovas and Frank Pfenning. *Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance*. Logical Methods in Computer Science 6(4), 2010; arXiv:1009.1861. https://arxiv.org/abs/1009.1861.

[16] Mario Carneiro. *Lean4Lean: Verifying a Typechecker for Lean, in Lean*. arXiv:2403.14064, 2024. https://arxiv.org/abs/2403.14064.